08-01 JSON-RPC 서비스 구현 분석

개요

Solana의 JSON-RPC 서비스는 외부 클라이언트가 블록체인과 상호작용할 수 있는 주요 인터페이스입니다. 이 서비스는 HTTP를 통해 JSON-RPC 2.0 프로토콜을 구현하여 계정 정보 조회, 트랜잭션 전송, 블록 데이터 조회 등의 기능을 제공합니다.

아키텍처 구조

핵심 컴포넌트

```
pub struct JsonRpcService {
thread_hdl: JoinHandle<()>,
request_processor: JsonRpcRequestProcessor,
close_handle: Option<CloseHandle>,
}
```

```
pub struct JsonRpcRequestProcessor {
bank_forks: Arc<RwLock<BankForks>>,
block_commitment_cache: Arc<RwLock<BlockCommitmentCache>>,
blockstore: Arc<Blockstore>,
config: JsonRpcConfig,
health: Arc<RpcHealth>,
```

```
cluster_info: Arc<ClusterInfo>,

// ... 기타 필드들

}
```

서비스 계층 구조

1. **JsonRpcService**: HTTP 서버 관리 및 요청 라우팅
2. **JsonRpcRequestProcessor**: 실제 RPC 메서드 구현 및 비즈니스 로직
3. **RPC Trait 모듈들**: 기능별로 분리된 RPC 인터페이스 정의

RPC 인터페이스 구조

기능별 RPC 모듈

Solana RPC는 기능에 따라 여러 모듈로 분리되어 있습니다:

1. Minimal RPC (`rpc_minimal`)

기본적인 블록체인 정보 조회 기능:

```
#[rpc]

pub trait Minimal {

type Metadata;



#[rpc(meta, name = "getBalance")]

fn get_balance(

&self,
```

```
meta: Self::Metadata,
pubkey_str: String,
config: Option<RpcContextConfig>,
) -> Result<RpcResponse<u64>>;


#[rpc(meta, name = "getEpochInfo")]
fn  get_epoch_info(
&self,
meta: Self::Metadata,
config: Option<RpcContextConfig>,
) -> Result<EpochInfo>;


#[rpc(meta, name = "getSlot")]
fn  get_slot(&self, meta: Self::Metadata, config: Option<RpcContextConfig>) -> R


#[rpc(meta, name = "getTransactionCount")]
fn  get_transaction_count(
&self,
meta: Self::Metadata,
config: Option<RpcContextConfig>,
) -> Result<u64>;
}
```

2. Bank Data RPC (`rpc_bank`)

은행(Bank) 관련 데이터 조회:

```
#[rpc]
pub trait BankData {
#[rpc(meta, name = "getInflationGovernor")]
fn get_inflation_governor(
&self,
meta: Self::Metadata,
config: Option<RpcContextConfig>,
) -> Result<RpcInflationGovernor>;

#[rpc(meta, name = "getMinimumBalanceForRentExemption")]
fn get_minimum_balance_for_rent_exemption(
&self,
meta: Self::Metadata,
data_len: usize,
config: Option<RpcContextConfig>,
) -> Result<u64>;
}
```

3. Full RPC (`rpc_full`)

고급 기능 및 트랜잭션 처리:

```
#[rpc]
pub trait Full {
#[rpc(meta, name = "sendTransaction")]
fn send_transaction(
&self,
meta: Self::Metadata,
data: String,
config: Option<RpcSendTransactionConfig>,
) -> Result<String>;


#[rpc(meta, name = "simulateTransaction")]
fn simulate_transaction(
&self,
meta: Self::Metadata,
data: String,
config: Option<RpcSimulateTransactionConfig>,
) -> Result<RpcResponse<RpcSimulateTransactionResult>>;


#[rpc(meta, name = "getBlock")]
fn get_block(
&self,
meta: Self::Metadata,
```

```
slot: Slot,
config: Option<RpcEncodingConfigWrapper<RpcBlockConfig>>,
) -> BoxFuture<Result<Option<UiConfirmedBlock>>>;
}
```

요청 처리 파이프라인

1. HTTP 요청 수신

```
impl JsonRpcService {
pub fn new(
rpc_addr: SocketAddr,
config: JsonRpcConfig,
// ... 기타 매개변수들
) -> Result<Self, String> {
// HTTP 서버 설정
let server = ServerBuilder::with_meta_extractor(
io,
move |req: &hyper::Request<hyper::Body>| {
// 요청별 메타데이터 추출
request_processor.clone()
},
)
```

```
.event_loop_executor(runtime.handle().clone())
.threads(1)
.cors(DomainsValidation::AllowOnly(vec![
AccessControlAllowOrigin::Any,
]))
.request_middleware(request_middleware)
.max_request_body_size(max_request_body_size)
.start_http(&rpc_addr);
}
}
```

2. 요청 미들웨어 처리

```
impl RequestMiddleware for RpcRequestMiddleware {
fn  on_request(&self, request: hyper::Request<hyper::Body>) -> RequestMiddleware
// 특수 경로 처리 (스냅샷 다운로드, 헬스 체크 등)
if  request.uri().path() == "/health" {
return hyper::Response::builder()
.status(hyper::StatusCode::OK)
.body(hyper::Body::from(self.health_check()))
.unwrap()
.into();
}
```

```
// 파일 다운로드 요청 처리
if self.is_file_get_path(request.uri().path()) {
return self.process_file_get(request.uri().path());
}
// 일반 JSON-RPC 요청은 그대로 전달
request.into()
}
}
```

3. JSON-RPC 메서드 디스패치

각 RPC 메서드는 `JsonRpcRequestProcessor` 의 해당 구현으로 라우팅됩니다:

```
impl JsonRpcRequestProcessor {
pub fn get_balance(
&self,
pubkey: &Pubkey,
config: RpcContextConfig,
) -> Result<RpcResponse<u64>> {
// 1. 설정에 따른 Bank 선택
let bank = self.get_bank_with_config(config)?;
// 2. 계정 잔액 조회
Ok(new_response(&bank, bank.get_balance(pubkey)))
}
```

```
pub fn get_account_info(
&self,
pubkey: &Pubkey,
config: Option<RpcAccountInfoConfig>,
) -> Result<RpcResponse<Option<UiAccount>>> {
// 설정 파싱
let RpcAccountInfoConfig {
encoding,
data_slice,
commitment,
min_context_slot,
} = config.unwrap_or_default();
// Bank 선택
let bank = self.get_bank_with_config(RpcContextConfig {
commitment,
min_context_slot,
})?;
// 계정 정보 인코딩 및 반환
let response = get_encoded_account(&bank, pubkey, encoding, data_slice, None)?;
Ok(new_response(&bank, response))
}
```

```
}
```

Bank 선택 메커니즘

Commitment Level에 따른 Bank 선택

```
impl JsonRpcRequestProcessor {
fn bank(&self, commitment: Option<CommitmentConfig>) -> Arc<Bank> {
let commitment = commitment.unwrap_or_default();
if commitment.is_confirmed() {
// 낙관적으로 확인된 Bank 사용
return self
.optimistically_confirmed_bank
.read()
.unwrap()
.bank
.clone();
}

// Commitment Level에 따른 슬롯 선택
let slot = self
.block_commitment_cache
.read()
```

```
.unwrap()
.slot_with_commitment(commitment.commitment);


// BankForks에서 해당 슬롯의 Bank 조회
let  r_bank_forks = self.bank_forks.read().unwrap();
r_bank_forks.get(slot).unwrap_or_else(|| {
// 슬롯을 찾을 수 없는 경우 루트 Bank 반환
warn!("Bank with {:?} not found at slot: {:?}", commitment.commitment, slot);
r_bank_forks.root_bank()
})
}
}
```

비동기 처리 및 성능 최적화

멀티스레드 런타임

```
// 멀티스레드 Tokio 런타임 생성
let  runtime = Arc::new(
tokio::runtime::Builder::new_multi_thread()
.worker_threads(rpc_threads)
.on_thread_start(move || renice_this_thread(rpc_niceness_adj).unwrap())
.thread_name("solRpcEl")
```

```
.enable_all()
.build()
.expect("Runtime"),
);
```

비동기 RPC 메서드

일부 RPC 메서드는 `BoxFuture` 를 반환하여 비동기 처리를 지원합니다:

```
#[rpc(meta, name = "getBlock")]
fn get_block(
&self,
meta: Self::Metadata,
slot: Slot,
config: Option<RpcEncodingConfigWrapper<RpcBlockConfig>>,
) -> BoxFuture<Result<Option<UiConfirmedBlock>>>;
```

캐싱 및 최적화

1. Largest Accounts Cache

```
let largest_accounts_cache = Arc::new(RwLock::new(LargestAccountsCache::new(
LARGEST_ACCOUNTS_CACHE_DURATION, // 2시간
)));
```

2. Prioritization Fee Cache

```
// 우선순위 수수료 정보 캐싱
let prioritization_fee_cache = Arc::new(PrioritizationFeeCache::default());
```

오류 처리

RPC 커스텀 오류

```
type RpcCustomResult<T> = std::result::Result<T, RpcCustomError>;

pub enum RpcCustomError {
BlockCleanedUp {
slot: Slot,
first_available_block: Slot,
},
BlockNotAvailable { slot: Slot },
MinContextSlotNotReached { context_slot: Slot },
// ... 기타 오류 유형들
}
```

오류 응답 생성

```
fn new_response<T>(bank: &Bank, value: T) -> RpcResponse<T> {
```

```
RpcResponse {
context: RpcResponseContext::new(bank.slot()),
value,
}
}
```

설정 및 구성

JsonRpcConfig

```
#[derive(Debug, Default, Clone)]
pub struct JsonRpcConfig {
pub enable_rpc_transaction_history: bool,
pub enable_extended_tx_metadata_storage: bool,
pub faucet_addr: Option<SocketAddr>,
pub health_check_slot_distance: u64,
pub rpc_bigtable_config: Option<RpcBigtableConfig>,
pub max_multiple_accounts: Option<usize>,
pub account_indexes: AccountSecondaryIndexes,
pub rpc_threads: usize,
pub rpc_niceness_adj: i8,
pub full_api: bool,
pub obsolete_v1_7_api: bool,
```

```
pub rpc_scan_and_fix_roots: bool,
pub max_request_body_size: Option<usize>,
pub disable_health_check: bool,
}
```

특수 기능

1. 스냅샷 다운로드

RPC 서비스는 HTTP GET을 통한 스냅샷 파일 다운로드를 지원합니다:

```
fn process_file_get(&self, path: &str) -> RequestMiddlewareAction {
let filename = match path {
DEFAULT_GENESIS_DOWNLOAD_PATH => {
self.ledger_path.join("genesis.tar.bz2")
}
_ => {
self.find_snapshot_file(stem)
}
};
// 파일 스트리밍 응답 생성
RequestMiddlewareAction::Respond {
should_validate_hosts: true,
response: Box::pin(async {
```

```
// 파일을 스트림으로 읽어서 응답
}),
}
}
```

2. 헬스 체크

```
fn health_check(&self) -> &'static str {
match self.health.check() {
RpcHealthStatus::Ok => "ok",
RpcHealthStatus::Behind { .. } => "behind",
RpcHealthStatus::Unknown => "unknown",
}
}
```

3. REST API 엔드포인트

일부 간단한 정보는 REST API로도 제공됩니다:

```
fn process_rest(bank_forks: &Arc<RwLock<BankForks>>, path: &str) -> Option<Stri
match path {
"/v0/circulating-supply" => {
let bank = bank_forks.read().unwrap().root_bank();
let total_supply = bank.capitalization();
```

```
let non_circulating_supply = calculate_non_circulating_supply(&bank)
.expect("Scan should not error on root banks")
.lamports;
Some(format!("{}", lamports_to_sol(total_supply - non_circulating_supply)))
}
"/v0/total-supply" => {
let bank = bank_forks.read().unwrap().root_bank();
Some(format!("{}", lamports_to_sol(bank.capitalization())))
}
_ => None,
}
}
```

보안 및 제한사항

1. 요청 크기 제한

```
pub const MAX_REQUEST_BODY_SIZE: usize = 50 * (1 << 10); // 50kB
```

2. CORS 설정

```
.cors(DomainsValidation::AllowOnly(vec![
AccessControlAllowOrigin::Any,
```

```
]))

.cors_max_age(86400)
```

3. Rate Limiting 및 리소스 보호

- 계정 조회 개수 제한: `MAX_MULTIPLE_ACCOUNTS`
- 프로그램 계정 필터 제한: `MAX_GET_PROGRAM_ACCOUNT_FILTERS`
- 서명 상태 조회 제한: `MAX_GET_SIGNATURE_STATUSES_QUERY_ITEMS`

결론

Solana의 JSON-RPC 서비스는 모듈화된 구조와 효율적인 캐싱, 비동기 처리를 통해 높은 성능과 확장성을 제공합니다. 다양한 commitment level 지원과 세밀한 설정 옵션을 통해 클라이언트의 다양한 요구사항을 충족할 수 있도록 설계되었습니다.